Phase 9 | Final Interview Bible & Cheat Sheet

High-signal revision for the day before an interview.

FastAPI Edition | Repository-grounded study material | Snapshot: 2026-10-24

5-minute project review

MetroInsights is a FastAPI planning prototype. It accepts a station/date/time; constructs 23 features; predicts hourly demand with HistGradientBoostingRegressor; creates 19 hourly points from 05:00 to 23:00; includes calendar context and a closed-hours gate; estimates service frequency using clearance, headway, fleet and signaling assumptions; and computes topology-based travel times with a line-expanded graph and Dijkstra. It has 243 station records and 11 project-defined route keys. Synthetic data has 3,361,176 rows. Chronological synthetic holdout: 438,615 rows, split at 2026-10-01. HGBR metrics: MAE 26.58, RMSE 51.10, R^2 0.9946, MAPE 5.18%. These are not observed-ridership metrics.

30-second pitch

“MetroInsights is a FastAPI transit-planning prototype that turns station, date, and time selections into hourly demand scenarios. It uses a 23-feature scikit-learn regressor trained on synthetic data, applies configurable train-capacity and headway assumptions, and estimates network travel times with a line-aware Dijkstra graph. I built the API, data/model pipeline, service-planning logic, and dashboard. The key caveat is that neither the demand labels nor several operating parameters are verified live DMRC data.”

Architecture in one line

Browser -> FastAPI/Uvicorn -> routes -> forecasting + calendar + operating hours + scheduler + graph -> CSV/JSON/joblib artifacts -> HTML/JSON.

Files to remember

- `run.py`: ASGI entry point.
- `app/__init__.py`: app/static setup.
- `app/routes.py`: endpoints and orchestration.
- `app/forecasting.py`: artifact load, features, prediction/profile.
- `app/scheduling_optimizer.py`: headway/capacity heuristic.
- `app/travel_graph.py`: mapping, edge construction, Dijkstra.
- `app/network_topology.py`: ordered line stops, aliases, interchanges, weights.
- `app/calendar_context.py`, `app/operating_hours.py`: time context/gating.
- `pipeline/03_train_and_evaluate_forecaster.py`: features, split, baselines, model.

Mapping concept in 60 seconds

The cleaned station table supplies canonical labels. The topology file supplies ordered station names along each route. Normalization/aliases translate topology labels into canonical station names. The graph creates a distinct node for each station-line pair, adds weighted edges between adjacent stops, and adds transfer edges between line nodes at an interchange. Dijkstra sums nonnegative edge times and finds the minimum route. Results are estimates because coordinates, fixed speed, fallback segment time, and single transfer penalty simplify operations.

Numbers and caveats

- 243 station rows: project cleaned table; not a verified official unique station count.
- 11 line/branch route keys: project topology grouping.
- 19 modeled hourly points, 05:00-23:00.
- 3,361,176 synthetic training records.
- 438,615 chronological synthetic holdout rows.
- HGBR: MAE 26.58, RMSE 51.10, R^2 0.9946, MAPE 5.18% on generated targets.
- Never call R^2 “accuracy”; never claim actual DMRC passenger-count validation.
- Travel destinations exclude the origin; count varies with graph coverage and selected source.

100 rapid-fire questions with compact answers

- What is it? A station/date/time scenario planning prototype.
- User? Transit planner/analyst.
- Live data? No.
- Backend? FastAPI.
- Server? Uvicorn.
- UI? Jinja, JS, CSS.
- ML library? scikit-learn.
- Data tools? Pandas and NumPy.
- Model? HistGradientBoostingRegressor.
- Feature count? 23.
- Target? Synthetic hourly passenger demand.
- Validation? Chronological out-of-time synthetic holdout.
- Split? 2026-10-01.
- Holdout rows? 438,615.
- Total rows? 3,361,176.
- MAE? 26.58 passengers/hour.
- RMSE? 51.10 passengers/hour.
- R^2? 0.9946.
- Is R^2 accuracy? No.
- Why synthetic caveat? Labels are generated, not observed.
- Baselines? Historical means and Ridge.
- Best baseline MAE? Ridge 53.67.
- Historical baseline MAE? 60.25.
- Why chronological split? Simulates later-date prediction.

- Leakage risk? Future or same-time targets in features.
- Cyclic feature? Sine/cosine period representation.
- Why two values? Preserve phase/quadrant.
- Historical feature source? Training-period aggregates.
- Does model train on API call? No.
- Artifact format? Joblib.
- Joblib trust? Load only trusted artifacts.
- Route graph package? Custom adjacency graph.
- Graph algorithm? Dijkstra.
- Dijkstra allows negative weights? No.
- Edge weights? Estimated minutes.
- Node? `(station,line)`.
- Why expanded? Explicit transfer states/cost.
- Interchange cost? Project-wide 5-minute assumption.
- Segment fallback? 2.5 minutes.
- Coordinate speed? 32 km/h assumption.
- Haversine? Great-circle distance formula.
- Route times official? No, estimates.
- Normalization purpose? Match variants of station labels.
- Alias purpose? Resolve known unmatched labels.
- Unmatched station behavior? Edge can be skipped.
- Improvement? Mapping audit and stable IDs.
- Why multi-source start? Begin on any source station line-state.
- Output includes origin? No.
- Complexity? O((V+E) log V).
- Why adjacency list? Efficient sparse graph traversal.
- API predict methods? GET query and POST JSON.
- Invalid date response? 400.
- Unknown station? Validation needs improvement.
- Pydantic body? Not currently used for predict contract.
- OpenAPI docs? `/api/docs`.
- HTML engine? Jinja2.
- Static assets? Starlette StaticFiles.
- Async means faster ML? No.
- Threadpool purpose? Keep sync work off event loop.
- GIL? Can limit Python CPU threading.
- Closed hours? Return zero/closed status.
- Profile slots? 19.
- Minute estimate? Linear interpolation.
- Is interpolation trained? No.

- Peak badge source? Demand quantiles/profile.
- Capacity utilization? Demand/clearance capacity.
- Occupancy? Not measured.
- Clearance? Space available for boarders after through-load assumption.
- Headway? Minutes between trains.
- Trains/hour relation? 60 divided by headway.
- Fleet constraint? Fleet size and turnaround.
- Signaling constraint? Minimum safe headway assumption.
- Through-load? Assumed already-onboard share.
- Interchange demand? Configured synthetic split.
- Real train count? No.
- DB? None currently.
- Static inputs? CSV/JSON.
- Why no DB? Read-only prototype inputs.
- When DB? Observations/concurrency/mutations.
- Startup loading? Eager module imports.
- Startup risk? Missing artifacts abort import.
- Worker memory? Each worker loads own copies.
- First optimization? Profile then batch 19 predictions.
- Caching target? Static graph/source routes.
- Retraining at startup? No.
- Queue present? No.
- Authentication? No.
- Rate limiting? Not in app code.
- Privacy data? Currently synthetic.
- Main model risk? Synthetic-to-real domain gap.
- Main routing risk? Incomplete/incorrect mapping.
- Main scheduler risk? Unverified assumptions.
- Testing priority? Unit/contract/smoke tests.
- Health endpoints? Add readiness/liveness.
- Monitoring? Add latency/error/model version.
- Next data step? Obtain quality observed counts.
- Next model step? Time-based real validation/calibration.
- Next API step? Typed request schema/validation.
- Next topology step? Verified stable IDs/GTFS.
- Final honesty point? Prototype, estimates, synthetic evidence.

50 difficult questions with concise answers

- Why is holdout high? Generated patterns may be smooth and learnable.
- How prove external validity? Independent observed time split and error analysis.

- Why HGBR vs XGBoost? Existing sklearn stack; compare options empirically.
- Why not normalize? Trees generally don't require scaling.
- Are integer categories ordinal? They can imply order; compare categorical/one-hot.
- Why use MAPE? Intuitive relative error; zero labels excluded, so caveat.
- Why is R^2 not accuracy? It is explained variance metric for regression.
- Could chronological split still leak? Yes, feature availability and generated rule leakage need audit.
- Why is historical baseline weak? Depends on synthetic history and sparse station-hour patterns.
- What if a category is unseen? Mapping/default behavior must be explicit.
- What is concept drift? Relationship between features and target changes over time.
- How monitor drift? Input/target distribution and delayed error metrics.
- What if holiday calendar wrong? Context features distort scenarios; version calendar.
- What if training and serving schema diverge? Contract tests/metadata versioning.
- Why feature order? Estimator expects trained column schema.
- What does batch inference improve? Fewer DataFrame/model calls and overhead.
- Why not async predict? sklearn is synchronous CPU work.
- Why threadpool not process? Simple nonblocking integration; process costs higher.
- When separate service? Independent model scaling/resource isolation is needed.
- What if worker OOM? Reduce workers/cache, quantify model copy, scale memory.
- How deploy artifact safely? Version/hash, trusted build, compatibility manifest.
- Is joblib safe from untrusted file? No, pickle-based loading is unsafe.
- Why Dijkstra not BFS? Weighted edges are not equal.
- Why not Bellman-Ford? No negative edges; heap Dijkstra is simpler/faster.
- Why line states? Transfer is a distinct action with cost.
- Is starting transfer free? Multi-source origins imply yes at source; document assumption.
- Could Dijkstra support time-dependent edges? Not standard static costs; use time-dependent routing methods.
- Why bidirectional edges? Simplifies prototype; actual directed operation may differ.
- How detect mapping issues? Compare topology set to canonical map and assert coverage.
- What causes silent reachability loss? Failed name mapping or omitted route segment.
- Why Haversine imperfect? Track geometry differs from straight line.
- Why max segment threshold? Catch obvious bad coordinate distances.
- Could threshold hide real long segments? Inspect official distances and thresholds.
- Why fallback duration? Keep edges traversable without reliable coordinates.
- Why constant transfer penalty weak? Stations differ in walking/vertical transfer time.
- What is clearance utilization? Demand relative to residual boarding offer per hour.
- Why peak still comfortable? Forecast low or modeled capacity high.
- Are capacity values official? Treat as project assumptions pending verification.
- Does schedule optimize fleet globally? No, heuristic per line/station/hour.
- Is wait estimate realistic? Headway/2 assumes random passenger arrivals and regular service.
- What is largest business risk? Misrepresenting estimates as operational truth.
- What protects input boundary? Basic parsing; typed/range/station validation needed.

- What about request flooding? Edge rate limits and concurrency controls.
- How avoid exposing internals? Stable error messages; sanitized logs.
- Why no DB transaction? No mutable relational workflow today.
- How add real data? Provenance schema, validation, aggregation, privacy controls.
- How handle missing observations? Distinguish missing from zero; impute or flag.
- How test closed hours? Parameterized boundary tests around 05:00/23:30.
- How test graph? Tiny hand-computable graph plus integration station coverage.
- What would you not claim? Real-world accuracy, live occupancy, official timetable recommendation.

Final 1-page checklist

[ ] Say prototype and synthetic data before metrics. [ ] Name 243 station records, not official total. [ ] Explain API flow. [ ] Explain features/model and chronological split. [ ] Explain mapping and line-expanded graph. [ ] Explain Dijkstra nonnegative weights. [ ] Distinguish capacity clearance from occupancy. [ ] State no live count/auth/database. [ ] Acknowledge runtime smoke verification status. [ ] Propose real data, tests, provenance and calibration.

How to use this series

Read Phase 1 first, then Phase 2 while opening the source files. Use Phases 3, 5, 6, and 7 to deepen technical understanding; rehearse with Phases 4 and 8; revise with Phase 9. The editable Markdown beside each PDF is the source of truth for corrections.

Evidence and honesty rules

MetroInsights is a planning prototype. Its demand targets and holdout are synthetic. Model metrics describe agreement with generated scenario labels, not real passenger counts or operational DMRC accuracy. Network topology, coordinates, capacities, headways, service frequency, and interchange splits are project data or configurable assumptions unless separately verified. Say “estimated” and explain the assumptions. Do not describe this as a live passenger-count, train occupancy, or production DMRC system.

Verified snapshot used in this edition

The cleaned station table contains 243 station records. The topology graph's destination count depends on the selected source and excludes the source itself; 242 destinations can be returned when all other modeled stations are connected. Do not say 226 or 228 without checking the current data. The project defines 11 route keys, including branch keys. The synthetic training series uses 243 stations x 728 dates x 19 hourly slots (05:00-23:00), or 3,361,176 rows. A chronological holdout begins 2026-10-01 and has 438,615 rows. The recorded HistGradientBoostingRegressor scores are MAE 26.58, RMSE 51.10, R2 0.9946, and MAPE 5.18%; Ridge MAE is 53.67 and historical station/day/hour mean MAE is 60.25. These are synthetic-only evaluation results.

Current stack and constraints

FastAPI + Uvicorn, Starlette/Jinja2, server-rendered HTML with JavaScript/CSS, Pandas, NumPy, scikit-learn, joblib, CSV/JSON, and a custom adjacency graph with heapq Dijkstra. No relational database, live ridership feed, authentication, or distributed job queue is present. The generated multi-million-row CSV is a training input; runtime inference loads compact serialized model/metadata plus station and operations inputs, rather than reading every training row on each request. Runtime startup imports routes and eagerly loads artifacts/builds the graph, so missing files can prevent startup. The current FastAPI migration has passed static compilation and diff checks in prior work, but an actual runtime smoke test was not possible in the offline environment; verify locally before claiming deployment verification.